L01, Group 06
Melody Yang 400307007
Cynthia Sun 400238765

McMaster University

MECHTRON 3TB4

Lab 3 Report

Date: March 13, 2026

Lab 3 Report

Overview

In this lab, two simple Digital-Signal-Processing (DSP) modules were created and implemented on the DE1-SoC board, which are the FIR filter (SW[0]) and an echo machine (SW[1]). They are both used to process the given L01_Group6.wav file to decrypt the secret message masked by a high frequency pitch at 2000 Hz, determined using the Fast-Fourier Transform (FFT) function in MATLAB in Tutorial 3. The FIR filter and echo machine were tested with other audio files when they were connected to the laptop to verify their functionality for processing audio from the hardware. Furthermore, feedback could be heard from the microphone as well when SW[2] was enabled, and selecting either DSPs allow for the user to either filter the audio (specifically the 2000 Hz noise), or hear a delayed echo. To increase the volume, press KEY[0], and if the counter reaches the maximum volume, then further presses of KEY[0] will decrease the volume until it is muted. This cycle is repeated whenever either the minimum or maximum is reached from pressing KEY[0]. To reset the volume to the original state, SW[3] was enabled.

Questions

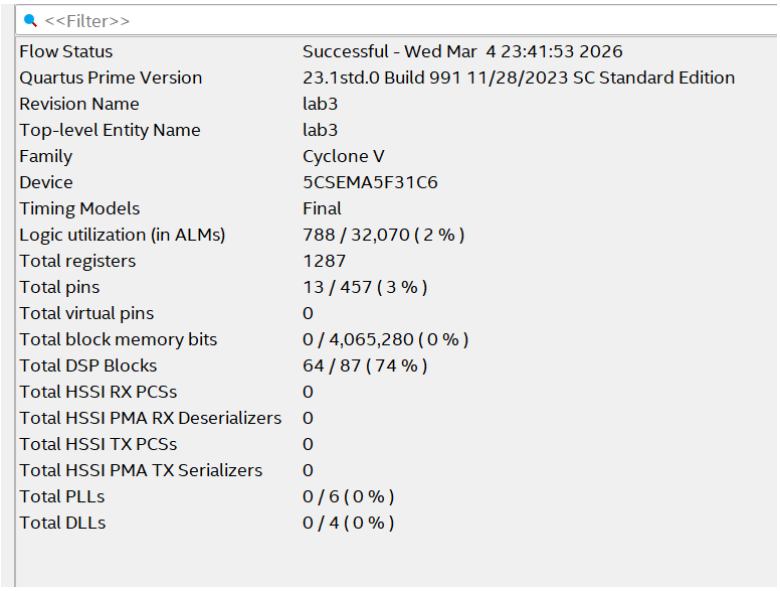
1. What values did you use for the echo delay and the division factor?

   The echo delay was 1920, which is 0.24 seconds (calculated using 64*30/8000). The division factor used was 4.

2. There are a limited number of multipliers on the Cyclone V FPGA. How can you redesign the filter to use less multiplier units?

   The filter can be redesigned using fewer multiplier units by exploiting the symmetry of the FIR coefficients. Currently, each of the taps is computed individually, but using symmetry, the pairs of delayed samples can be added first and then multiplied by a single shared coefficient, which reduces the number of multipliers by half. In the case that the total taps is 65 based on our selected parameter value, then this could reduce the multiplier requirement from 65 to 33. Another potential way to limit the number of multipliers is to ensure all the coefficients are a power of two which would allow the entire operation to be bit-shifted instead which won't use any multipliers. However, this can sacrifice the FIR filter performance and accuracy.

3. Open the compilation report in Quartus, and report the following numbers: Total number of logic elements used by your circuit, total number of registers, total number of pins, and the maximum number of logic elements that can fit on the FPGA you used.



<<Filter>>	
Flow Status	Successful - Wed Mar 4 23:41:53 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab3
Top-level Entity Name	lab3
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	788 / 32,070 (2 %)
Total registers	1287
Total pins	13 / 457 (3 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	64 / 87 (74 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 1. Compilation report of the Top Level Module.

Table 1. Summary for Components for the Top Level Module in Lab 3.

Feature	**Quantity**
Total number of logic elements	788
Total number of registers	1287
Total number of pins	13
Max number of logic elements that can fit on FPGA	32,070

4. What is a logic element, and what are the components that make up a logic element on the Cyclone V FPGA?

   A logic element is the basic programmable building block of an FPGA and is used to implement digital logic functions such as combinational logic (gates such as AND, OR, XOR, etc) and sequential logic (flip-flops, latches, etc). The

components that make up Cyclone V FPGA are an Adaptive Logic Module (ALM). Each ALM has 8 input look-up-tables (LUTs), dedicated carry chain logic for fast arithmetic operations, and four registers (flip-flops).

5. What is the purpose of the multiplexers in the lab design?

   The multiplexer allows the system to select which signal is passed to the output, which includes echo, unfiltered and FIR filtered output signals. The switches SW[0] and SW[1] are used to control which signal is selected for the output, which is then passed to the DSP subsystem. It makes debugging easier and allows the user to clearly hear the difference between the filtered, unfiltered, and echo signals.

6. How many memory bits does it take to store the samples in the delay circuit (shift register) in your echo machine?

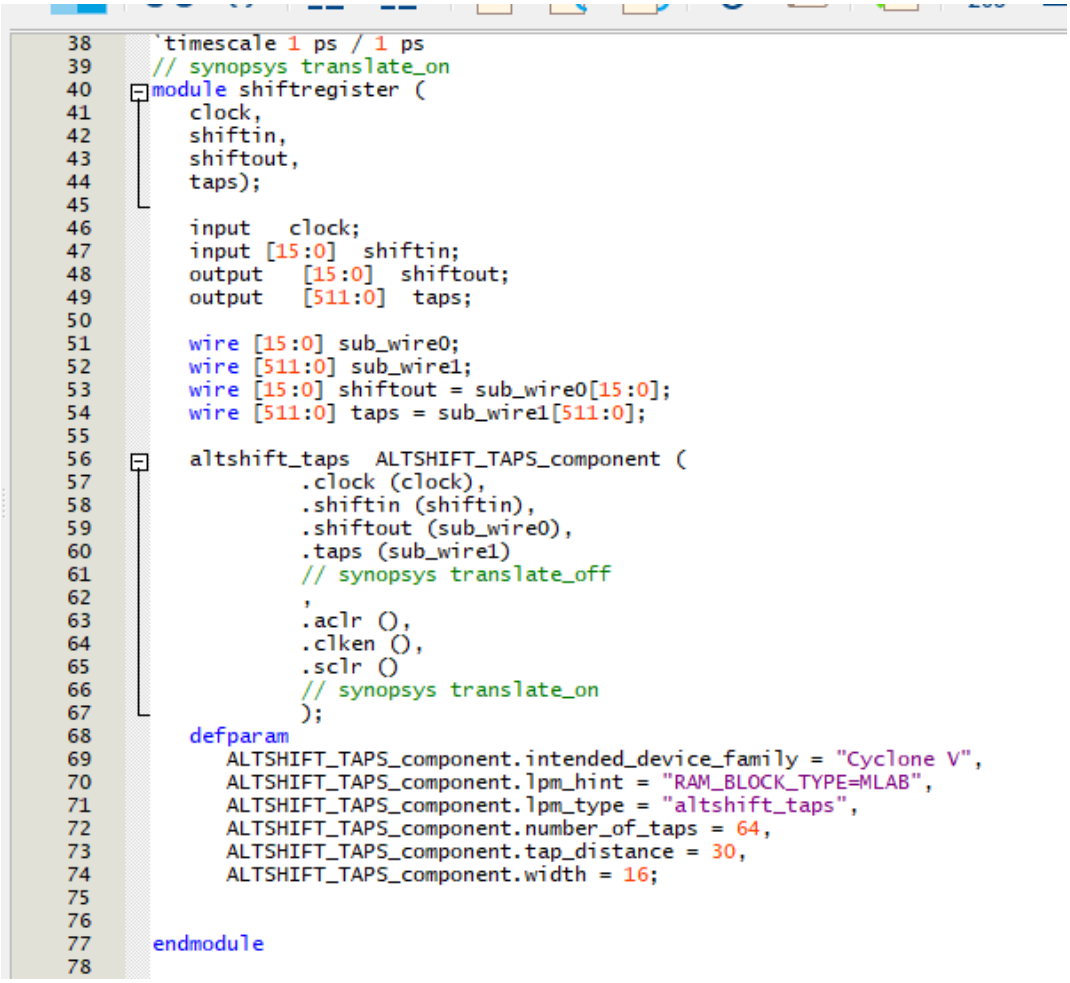
   The delay consisted of 1920 samples, and each sample has 16 bits. Therefore, the number of memory bits required is 30,720 bits (1920*16).

7. Considering that each logic block contains only one register, would your design have fit onto the FPGA had you not used the memory-based shift register?

   The total number of logical blocks available on the FPGA board is 32,070. Since the number of memory bits used in this design is 30,720, this means that the design would fit onto the FPGA even if memory-based shift registers weren't used. If the logical blocks required for the top-level module were added (788), the total of logical blocks used altogether is 31,508, which means that the current design would fit onto the FPGA. However, if a longer delay was used, then the design would not fit onto the FPGA had we not used the memory-based shift register.

Shift Register

The shift-register was used in the echo machine module generated by Quartus Prime using the IP Catalog > Library > On Chip Memory > Shift Register (RAM-based). The shift register uses the inputs to create a stored delayed list. In this case, the number of taps used for the shift-register was adjusted and tested using the DE1-SoC FPGA board to check for the best echo effect, resulting in the number_of_taps to be 64, the tap_distance to be 30 samples between taps, and the tap width to be 16-bits. This is used to generate a delayed list of inputs that will be instantiated in the echo_machine to create an echo effect.



```
38  `timescale 1 ps / 1 ps
39  // synopsys translate_on
40  module shiftregister (
41      clock,
42      shiftin,
43      shiftout,
44      taps);
45
46      input   clock;
47      input [15:0]  shiftin;
48      output   [15:0]  shiftout;
49      output   [511:0]  taps;
50
51      wire [15:0] sub_wire0;
52      wire [511:0] sub_wire1;
53      wire [15:0] shiftout = sub_wire0[15:0];
54      wire [511:0] taps = sub_wire1[511:0];
55
56      altshift_taps  ALTSHIFT_TAPS_component (
57                .clock (clock),
58                .shiftin (shiftin),
59                .shiftout (sub_wire0),
60                .taps (sub_wire1)
61                // synopsys translate_off
62                ,
63                .aclr (),
64                .clken (),
65                .sclr ()
66                // synopsys translate_on
67                );
68      defparam
69          ALTSHIFT_TAPS_component.intended_device_family = "Cyclone V",
70          ALTSHIFT_TAPS_component.lpm_hint = "RAM_BLOCK_TYPE=MLAB",
71          ALTSHIFT_TAPS_component.lpm_type = "altshift_taps",
72          ALTSHIFT_TAPS_component.number_of_taps = 64,
73          ALTSHIFT_TAPS_component.tap_distance = 30,
74          ALTSHIFT_TAPS_component.width = 16;
75
76
77  endmodule
78
```

Figure 2. Screenshot of the Shift Register module.

Echo Machine

The echo_machine.v module generates an echo effect by combining the current audio sample with the delayed version of the same signal. The input signal is x_in, which is then passed through the shift register. The shift register stores the previous samples and outputs a delayed_sample. The delayed signal is then scaled by shifting it two bits (>>>2), which divides the amplitude by 4 (2-bits = 2^2 = 4), which reduces the echo intensity and prevents excessive signal amplification. The scaled delay signal scaled_echo is added to the original input signal using a built-in adder within the module with a wider bit-width of 19-bits to handle overflow during the addition. If reset is not active, the results are truncated back to 16-bits and stored in the output register “y_out” on the rising edge of the clock. If reset is activated, y_out is cleared to 0.

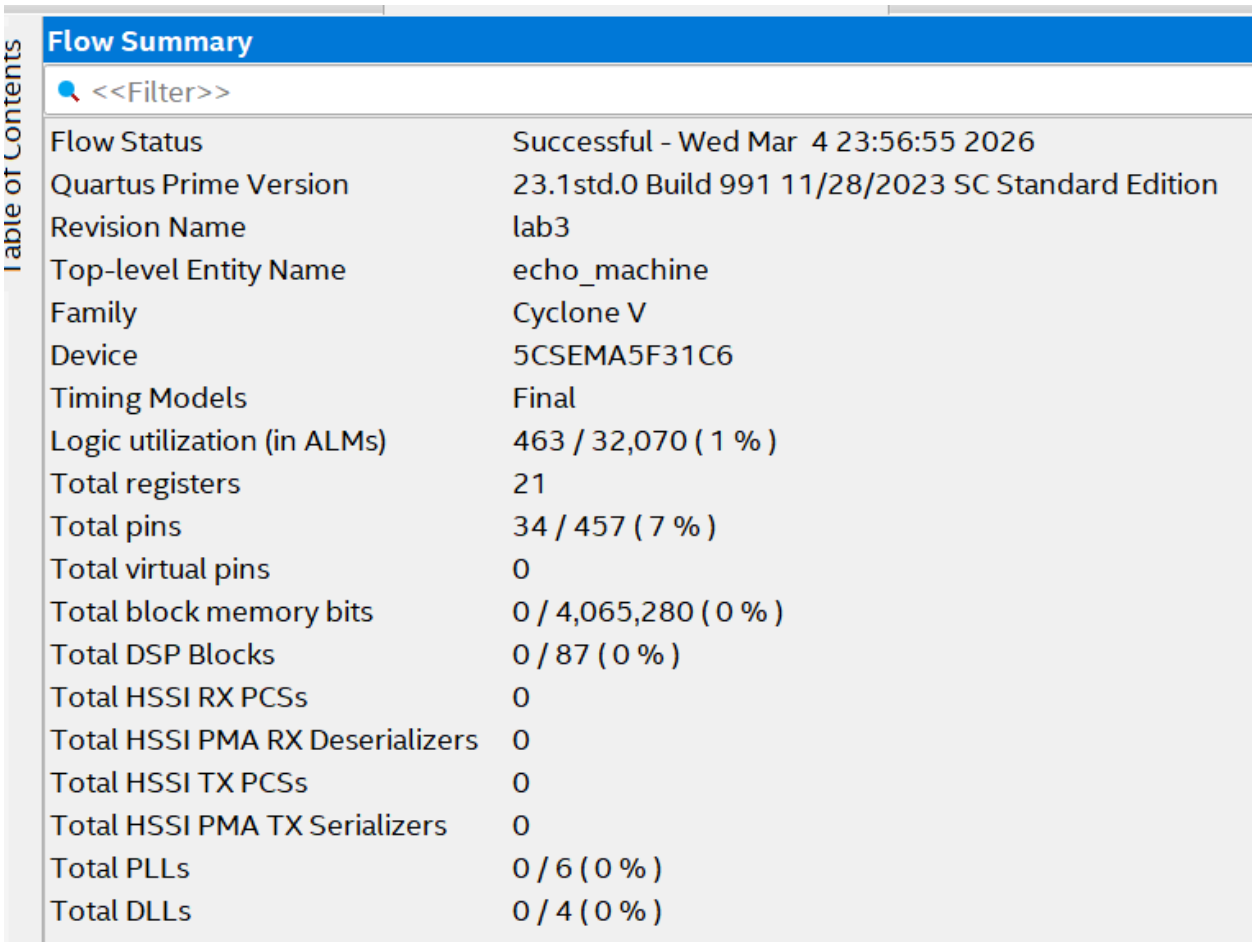


Figure 3. Compilation report of the Echo Machine module.

Table 2. Summary for Components for the Echo Machine in Lab 3.

Feature	Quantity
Total number of logic elements	463
Total number of registers	21
Total number of pins	34
Max number of logic elements that can fit on FPGA	32,070

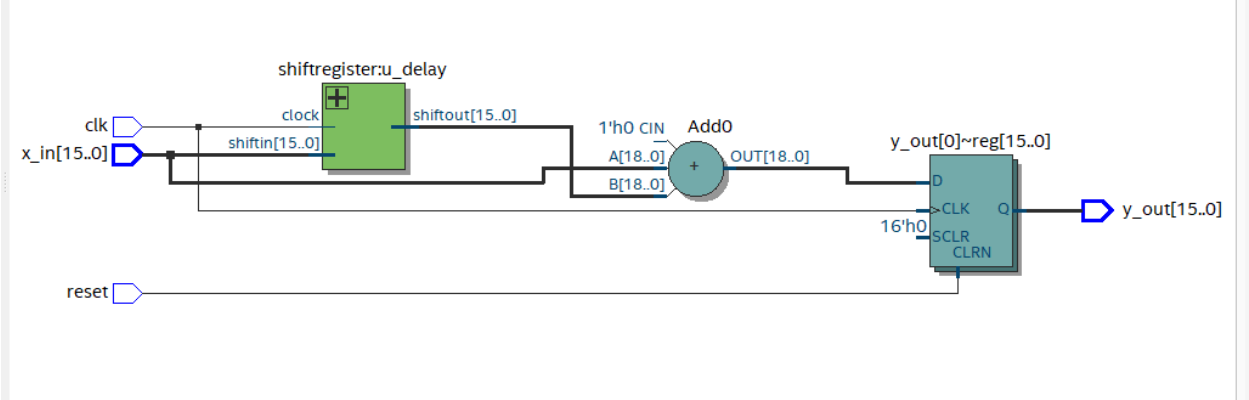
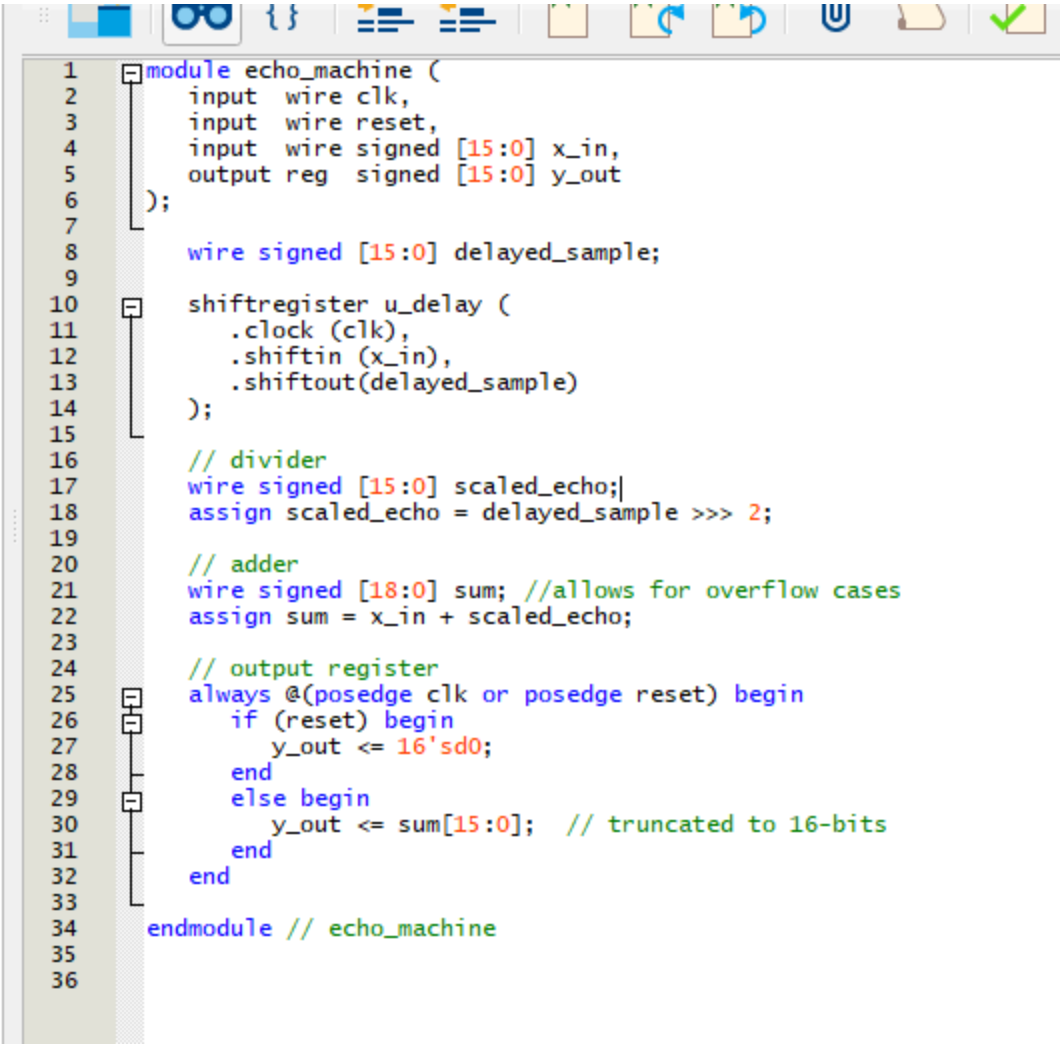


Figure 4. Screenshot of the implemented Echo Machine module.



```
1  module echo_machine (
2     input  wire clk,
3     input  wire reset,
4     input  wire signed [15:0] x_in,
5     output reg  signed [15:0] y_out
6  );
7
8     wire signed [15:0] delayed_sample;
9
10    shiftregister u_delay (
11       .clock (clk),
12       .shiftin (x_in),
13       .shiftout(delayed_sample)
14    );
15
16    // divider
17    wire signed [15:0] scaled_echo;
18    assign scaled_echo = delayed_sample >>> 2;
19
20    // adder
21    wire signed [18:0] sum; //allows for overflow cases
22    assign sum = x_in + scaled_echo;
23
24    // output register
25    always @(posedge clk or posedge reset) begin
26       if (reset) begin
27          y_out <= 16'sd0;
28       end
29       else begin
30          y_out <= sum[15:0];  // truncated to 16-bits
31       end
32    end
33
34 endmodule // echo_machine
35
36
```

Figure 5. Screenshot of the Echo Machine (full code in Code Section of report).

Multiplier

The multiplier used in the FIR module was generated by Quartus Prime using the IP Catalog > Library > Basic Function > Arithmetic. The multiplier used 2 inputs at 16-bits and created a product of 32-bits rather than the recommendation in the lab manual. This was done in order to ensure there was no data loss after using the multiplier, since it still needed to be summed up. Therefore, this method was used to achieve a more accurate result. Since it is 32-bits, it was truncated to 16-bits in the output by setting it as “y_out<=sum[15:0].” This ensures the data range is correct.

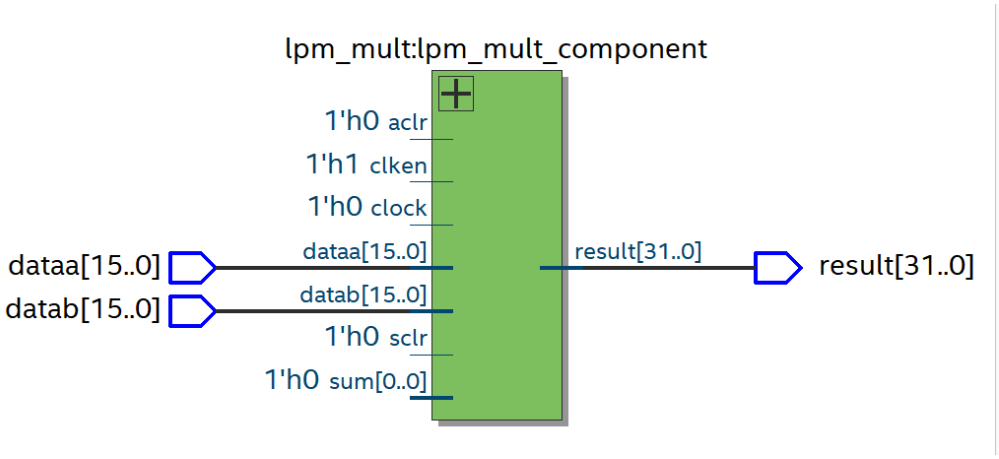
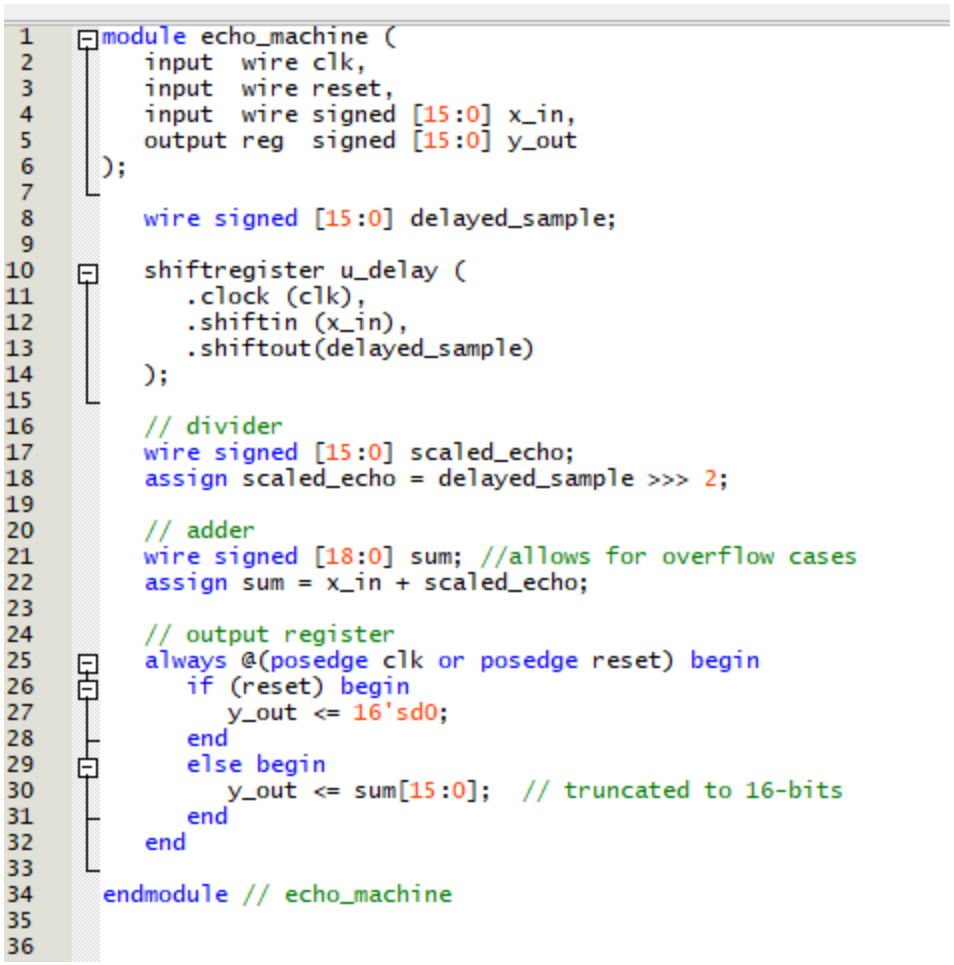


Figure 6. Screenshot of the implemented Multiplier module.



```
module echo_machine (
    input  wire clk,
    input  wire reset,
    input  wire signed [15:0] x_in,
    output reg  signed [15:0] y_out
);

    wire signed [15:0] delayed_sample;

    shiftregister u_delay (
        .clock (clk),
        .shiftin (x_in),
        .shiftout(delayed_sample)
    );

    // divider
    wire signed [15:0] scaled_echo;
    assign scaled_echo = delayed_sample >>> 2;

    // adder
    wire signed [18:0] sum; //allows for overflow cases
    assign sum = x_in + scaled_echo;

    // output register
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            y_out <= 16'sd0;
        end
        else begin
            y_out <= sum[15:0];  // truncated to 16-bits
        end
    end

endmodule // echo_machine

```

Figure 7. Screenshot of the Multiplier (full code in Code Section of report).

FIR Filter

The FIR filter module processes the audio samples using a finite impulse response filtering structure. The samples are stored in x_delay implemented as a shift register to preserve the current and previous input values. Each delayed sample is multiplied by the corresponding filter coefficient generated from the coefficients.txt file from MATLAB and scaled for fixed-point representation. A for loop adder is used to sum all the products together, initialized at 32-bits to ensure there is no overflow. The sum is later on scaled to fit within 16 bits by “>>> 15” operation. The final register is output per clock cycle when the reset is not 0. The total coefficients are set to 65 as well, based on the MATLAB parameter filter_len = 64. The parameter can be adjusted for any variable below 65.

Flow Summary

<<Filter>>

Flow Status	Successful - Wed Mar 4 21:17:54 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab3
Top-level Entity Name	fir_filter
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	257 / 32,070 (< 1 %)
Total registers	1141
Total pins	34 / 457 (7 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	64 / 87 (74 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 8. Compilation report of the FIR Filter module.

Table 3. Summary for Components for the FIR Filter in Lab 3.

Feature	Quantity
Total number of logic elements	257
Total number of registers	1141
Total number of pins	34
Max number of logic elements that can fit on FPGA	32,070

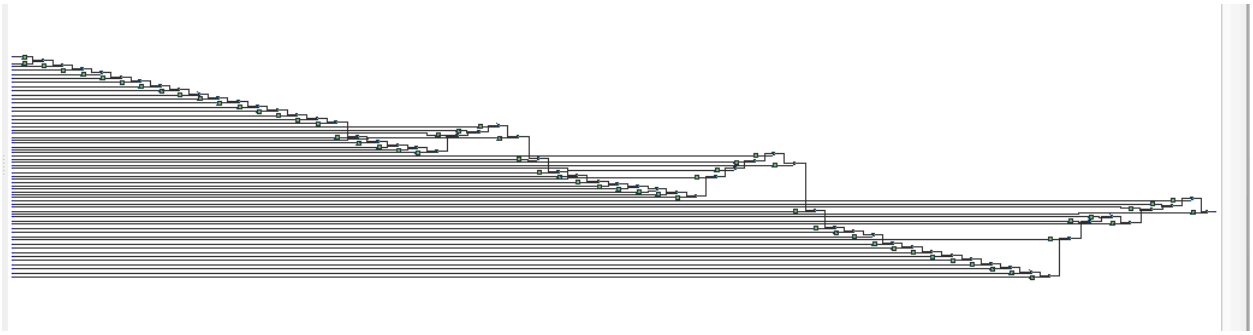
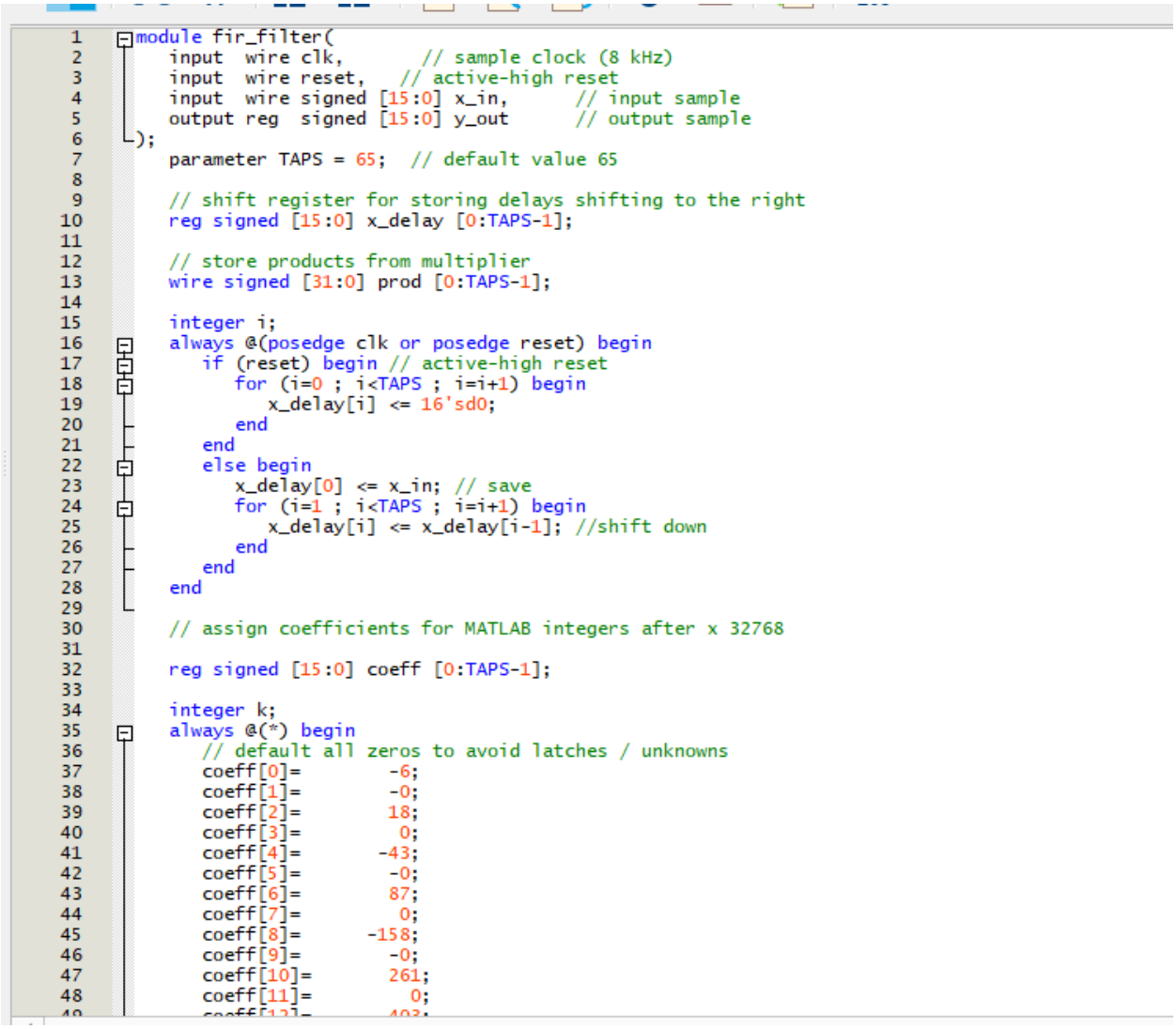


Figure 9. Screenshot of the implemented FIR Filter module.



```
module fir_filter(
    input  wire clk,       // sample clock (8 kHz)
    input  wire reset,   // active-high reset
    input  wire signed [15:0] x_in,      // input sample
    output reg  signed [15:0] y_out      // output sample
);
    parameter TAPS = 65;  // default value 65

    // shift register for storing delays shifting to the right
    reg signed [15:0] x_delay [0:TAPS-1];

    // store products from multiplier
    wire signed [31:0] prod [0:TAPS-1];

    integer i;
    always @(posedge clk or posedge reset) begin
       if (reset) begin // active-high reset
          for (i=0 ; i<TAPS ; i=i+1) begin
             x_delay[i] <= 16'sd0;
          end
       end
       else begin
          x_delay[0] <= x_in; // save
          for (i=1 ; i<TAPS ; i=i+1) begin
             x_delay[i] <= x_delay[i-1]; //shift down
          end
       end
    end

    // assign coefficients for MATLAB integers after x 32768

    reg signed [15:0] coeff [0:TAPS-1];

    integer k;
    always @(*) begin
       // default all zeros to avoid latches / unknowns
       coeff[0]=        -6;
       coeff[1]=        -0;
       coeff[2]=        18;
       coeff[3]=         0;
       coeff[4]=       -43;
       coeff[5]=        -0;
       coeff[6]=        87;
       coeff[7]=         0;
       coeff[8]=      -158;
       coeff[9]=        -0;
       coeff[10]=       261;
       coeff[11]=         0;
```

Figure 10. Screenshot of the FIR Filter (full code in Code Section of report).

Multiplexer

The 3-to-1 Multiplexer is used to switch between generating an audio of the original signal, the signal after the FIR filter is applied, or an echo effect using the echo machine. A minimum of 2-bit input was required for switching between 3 different cases. The original sound is set at 2'b00, the FIR filter is set at 2'b01, and the echo machine is set at 2'b10, with the default set as the original signal.

Flow Summary

<<Filter>>

Flow Status	Successful - Wed Mar 4 21:09:56 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab3
Top-level Entity Name	mux3_1
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	10 / 32,070 (< 1 %)
Total registers	0
Total pins	66 / 457 (14 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 11. Compilation report of the Multiplexer module.

Table 4. Summary for Components for the Multiplexer in Lab 3.

Feature	**Quantity**
Total number of logic elements	10
Total number of registers	0
Total number of pins	66
Max number of logic elements that can fit on FPGA	32,070

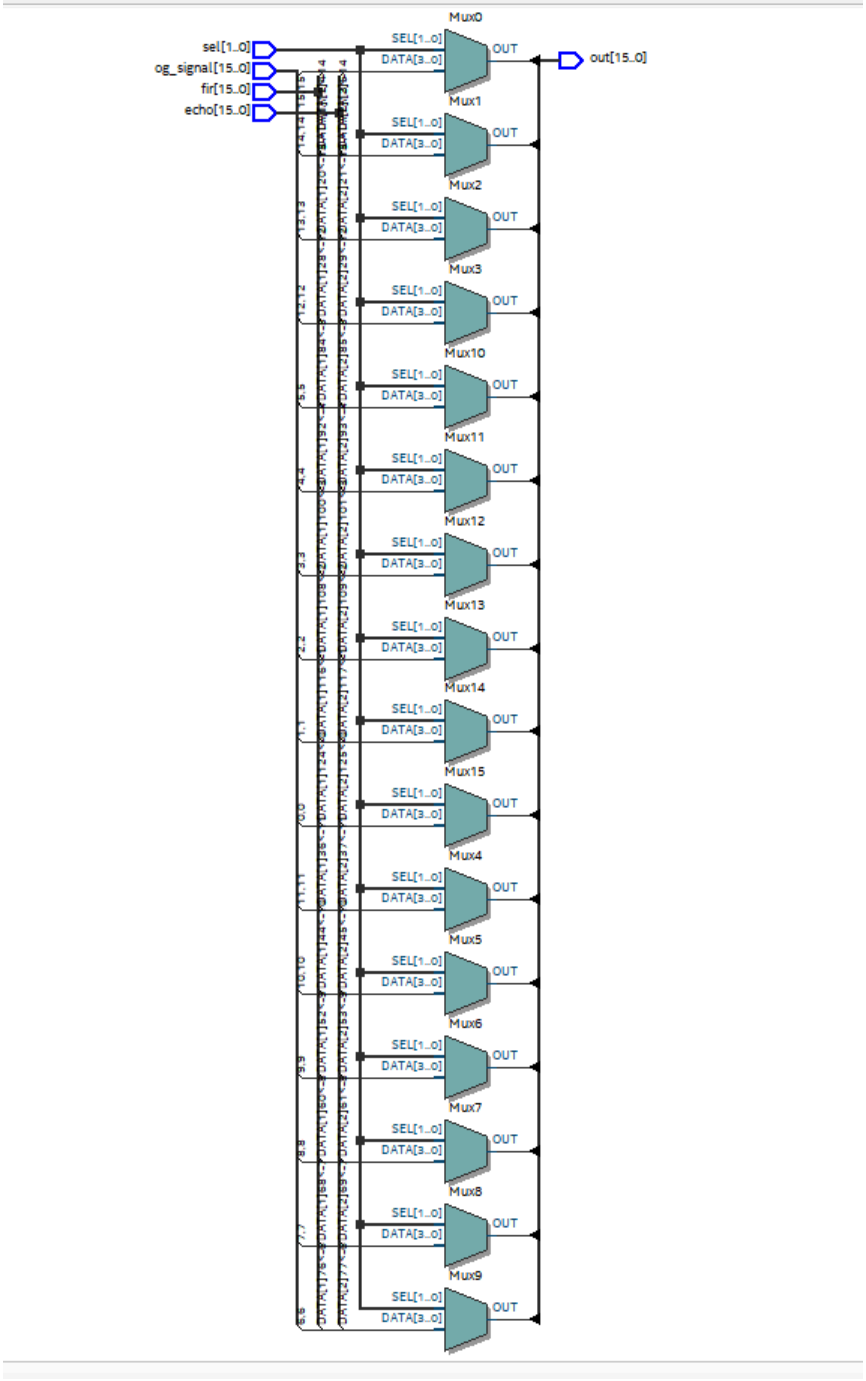
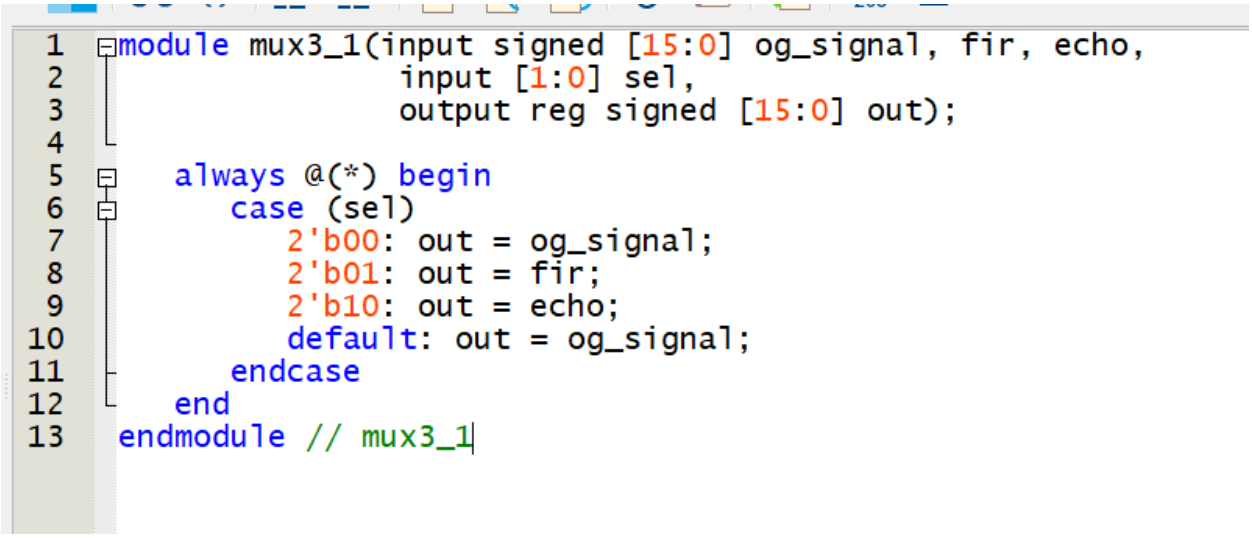


Figure 12. Screenshot of the implemented Multiplexer module.

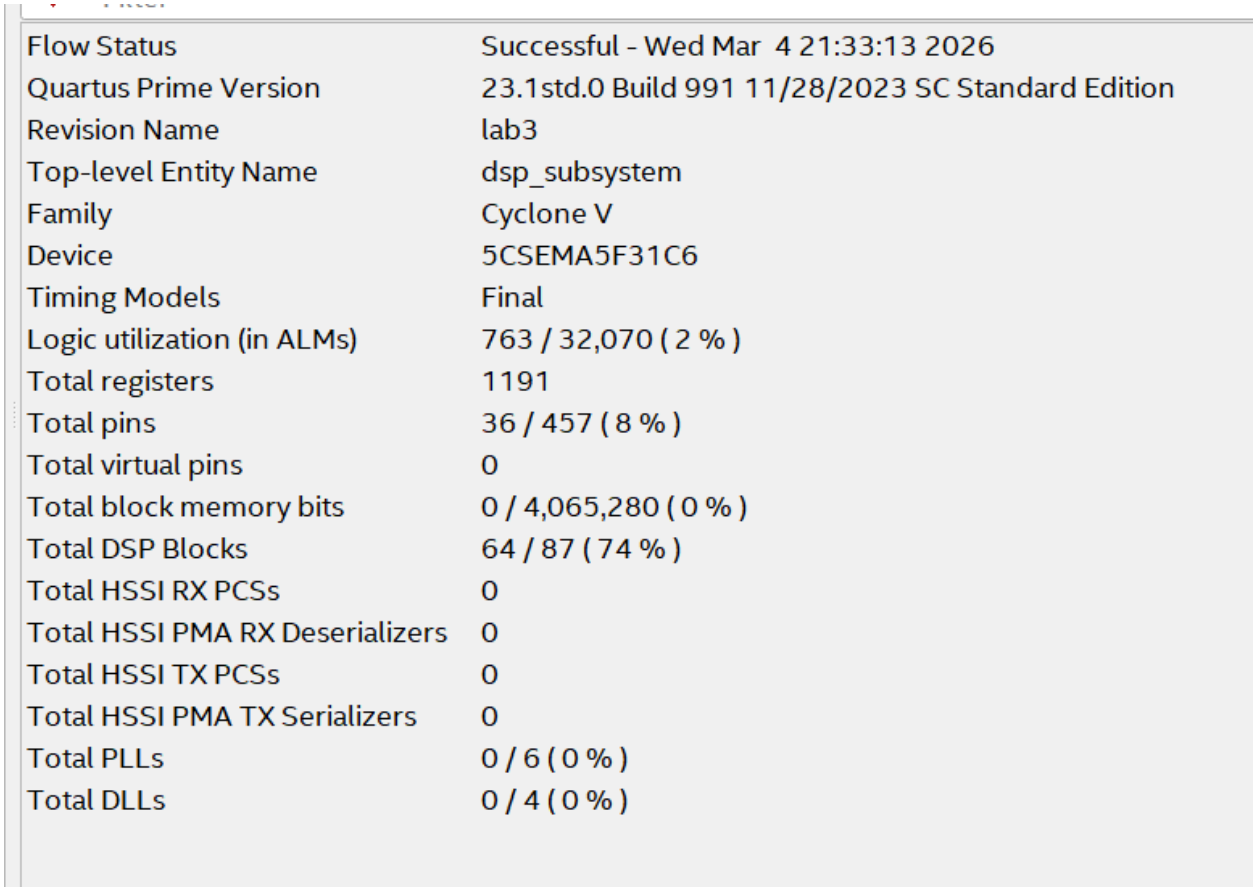


```
1  module mux3_1(input signed [15:0] og_signal, fir, echo,
2                input [1:0] sel,
3                output reg signed [15:0] out);
4
5     always @(*) begin
6        case (sel)
7           2'b00: out = og_signal;
8           2'b01: out = fir;
9           2'b10: out = echo;
10          default: out = og_signal;
11       endcase
12    end
13 endmodule // mux3_1
```

Figure 13. Screenshot of the Multiplexer (full code in Code Section of report).

DSP Subsystem

The DSP subsystem module is used to instantiate the fir_filter.v, echo_machine.v and the mux3_1.v modules which connect it to the top-level module lab3.v. The inputs received in this module contain all the signals required by the three instantiated modules. The input signal is sent in parallel to the FIR filter, the echo machine and directly to the multiplexer as the original signal. The 3-to-1 multiplexer chooses between the original, filtered, or echoed signal based on the selector input. The selected signal is assigned to the output_sample and returned to the top-level module as the audio output.



Flow Status	Successful - Wed Mar 4 21:33:13 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab3
Top-level Entity Name	dsp_subsystem
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	763 / 32,070 (2 %)
Total registers	1191
Total pins	36 / 457 (8 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	64 / 87 (74 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 14. Compilation report of the DSP Subsystem module.

Table 5. Summary for Components for the DSP Subsystem in Lab 3.

Feature	Quantity
Total number of logic elements	763
Total number of registers	1191
Total number of pins	36
Max number of logic elements that can fit on FPGA	32,070

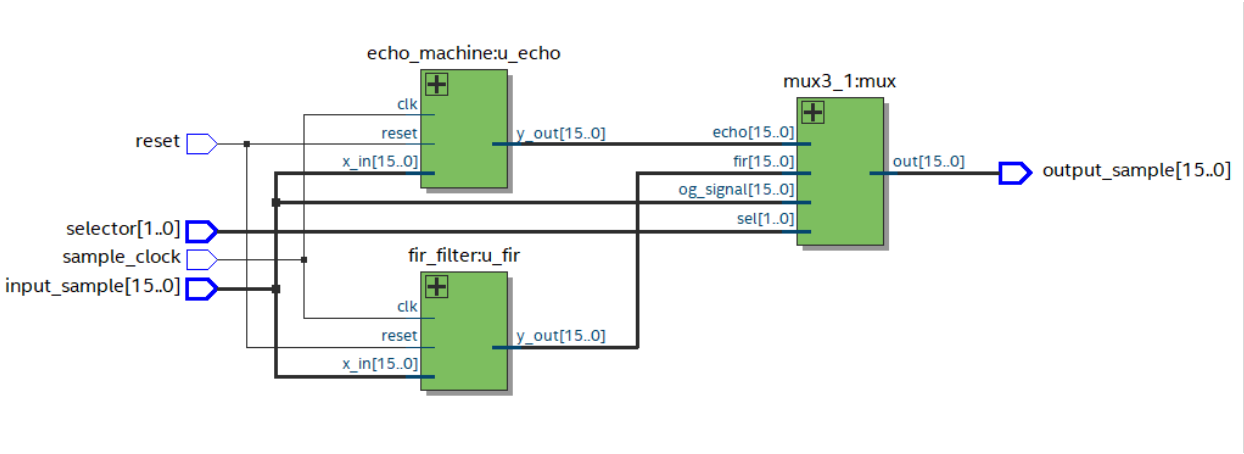
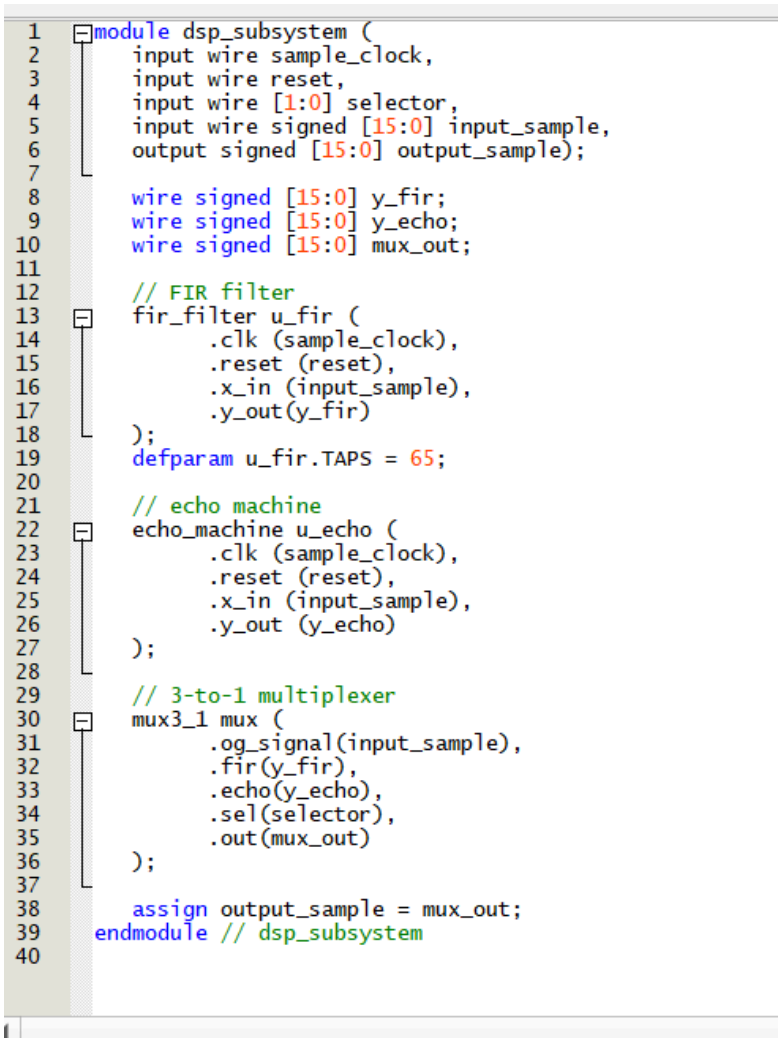


Figure 15. Screenshot of the implemented DSP Subsystem module.



```
module dsp_subsystem (
    input wire sample_clock,
    input wire reset,
    input wire [1:0] selector,
    input wire signed [15:0] input_sample,
    output signed [15:0] output_sample);

    wire signed [15:0] y_fir;
    wire signed [15:0] y_echo;
    wire signed [15:0] mux_out;

    // FIR filter
    fir_filter u_fir (
          .clk (sample_clock),
          .reset (reset),
          .x_in (input_sample),
          .y_out(y_fir)
    );
    defparam u_fir.TAPS = 65;

    // echo machine
    echo_machine u_echo (
          .clk (sample_clock),
          .reset (reset),
          .x_in (input_sample),
          .y_out (y_echo)
    );

    // 3-to-1 multiplexer
    mux3_1 mux (
          .og_signal(input_sample),
          .fir(y_fir),
          .echo(y_echo),
          .sel(selector),
          .out(mux_out)
    );

    assign output_sample = mux_out;
endmodule // dsp_subsystem

```

Figure 16. Screenshot of the DSP Subsystem (full code in Code Section of report).

Top Level Module

The top-level module was provided, and no alterations were made. The top-level module is able to interact with the rest of the modules we created without changing them since it instantiates dsp_subsystem.v. This is possible since the two DSP and multiplexer modules are instantiated within it.

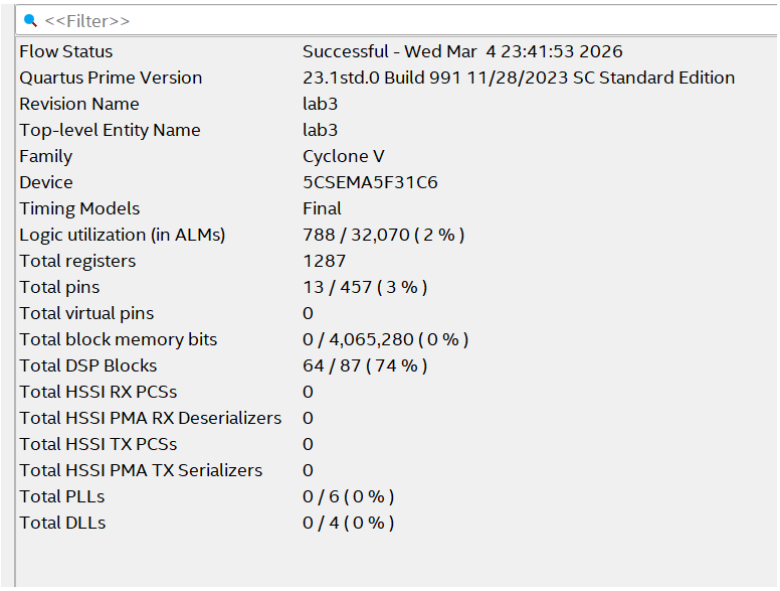


Figure 17. Compilation report of the Top Level Module.

Table 6. Summary for Components for the Top Level Module in Lab 3.

Feature	Quantity
Total number of logic elements	788
Total number of registers	1287
Total number of pins	13
Max number of logic elements that can fit on FPGA	32,070

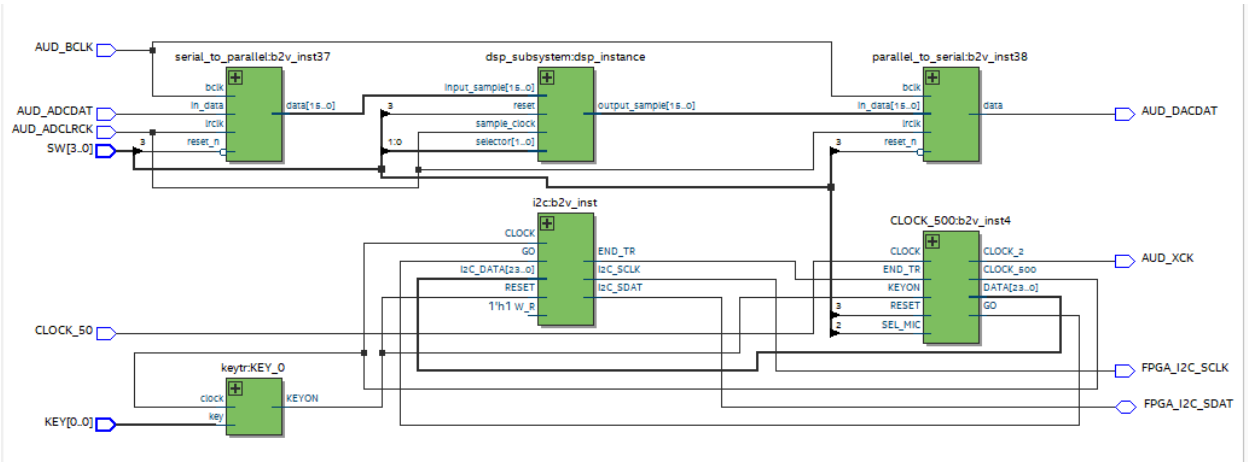


Figure 18. RTL view of the circuit.

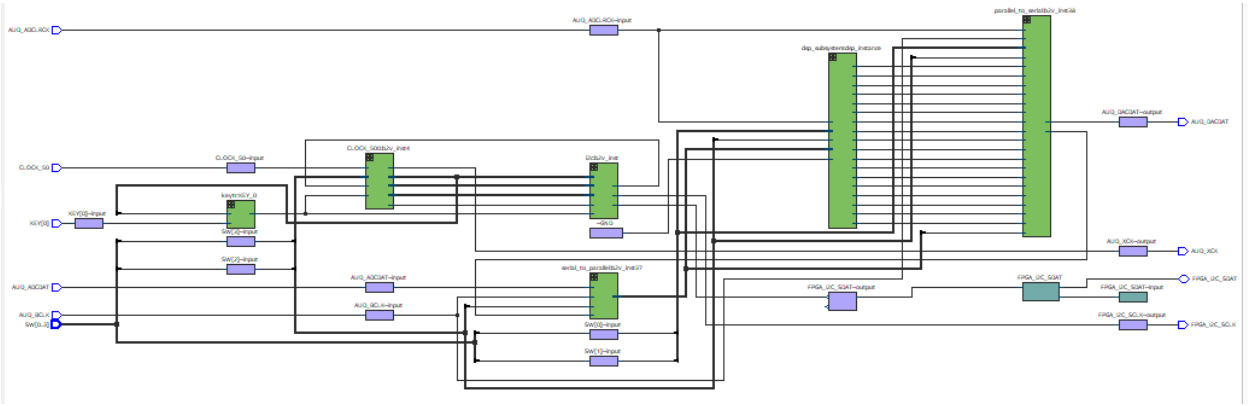


Figure 19. Technology map viewer (post-mapping) of the circuit.

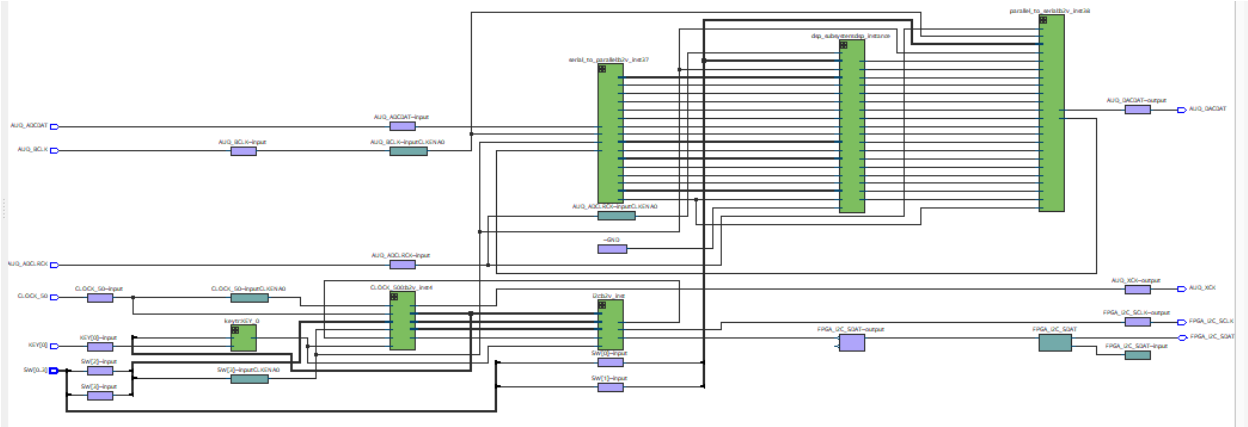


Figure 20. Technology map viewer (post-fitting) of the circuit.

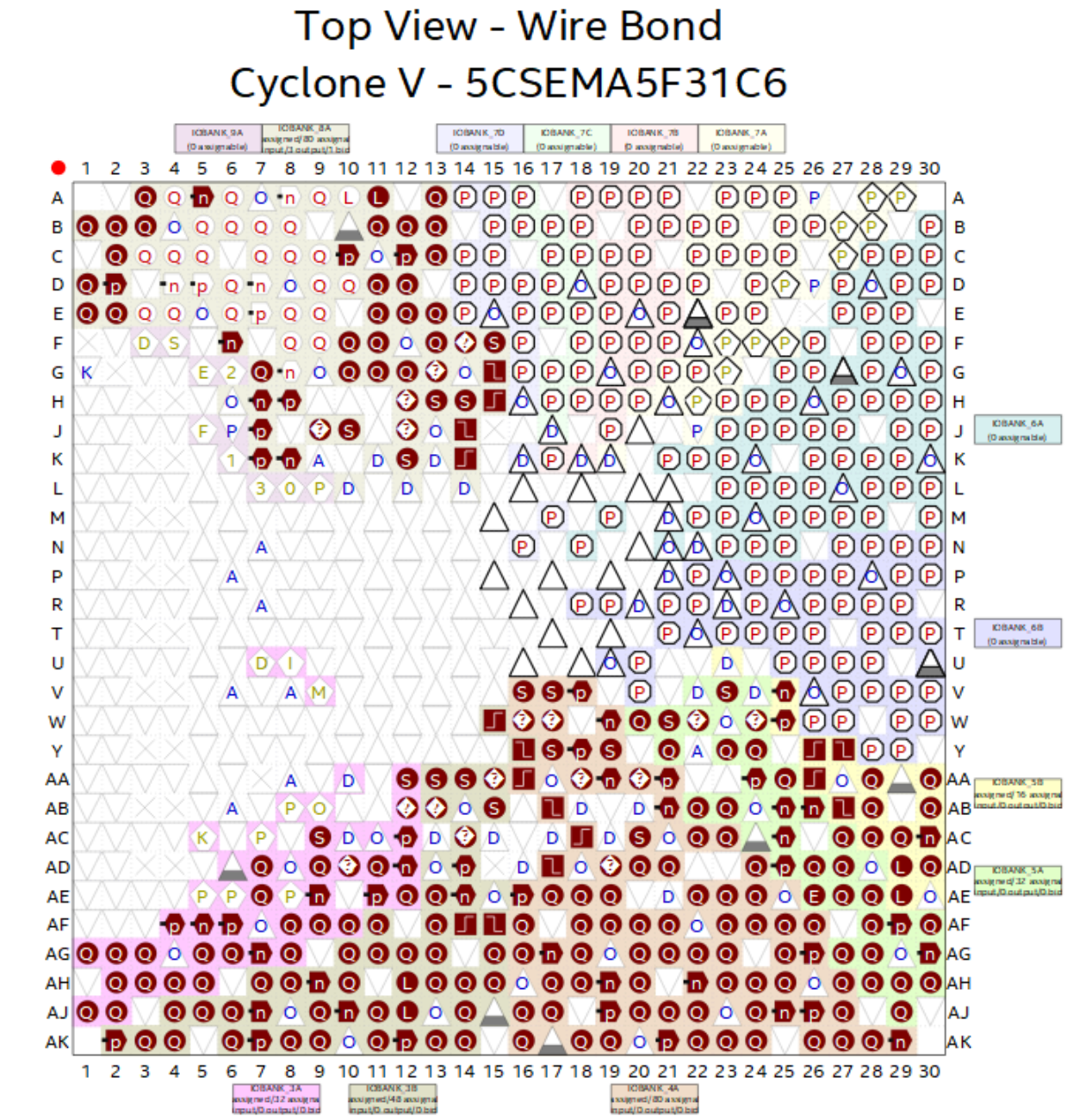


Figure 21a. Pin planner image.

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength
AUD_ADCDAT	Input	PIN_K7	8A	B8A_N0	PIN_K7	2.5 V		12mA (default)
AUD_ADCLRCK	Input	PIN_K8	8A	B8A_N0	PIN_K8	2.5 V		12mA (default)
AUD_BCLK	Input	PIN_H7	8A	B8A_N0	PIN_H7	2.5 V		12mA (default)
AUD_DACDAT	Output	PIN_J7	8A	B8A_N0	PIN_J7	2.5 V		12mA (default)
AUD_XCK	Output	PIN_G7	8A	B8A_N0	PIN_G7	2.5 V		12mA (default)
CLOCK_50	Input	PIN_AF14	3B	B3B_N0	PIN_AF14	2.5 V		12mA (default)
FPGA_I2C_SCLK	Output	PIN_J12	8A	B8A_N0	PIN_J12	2.5 V		12mA (default)
FPGA_I2C_SDAT	Bidir	PIN_K12	8A	B8A_N0	PIN_K12	2.5 V		12mA (default)
KEY[0]	Input	PIN_AA14	3B	B3B_N0	PIN_AA14	2.5 V		12mA (default)
SW[3]	Input	PIN_AF10	3A	B3A_N0	PIN_AF10	2.5 V		12mA (default)
SW[2]	Input	PIN_AF9	3A	B3A_N0	PIN_AF9	2.5 V		12mA (default)
SW[1]	Input	PIN_AC12	3A	B3A_N0	PIN_AC12	2.5 V		12mA (default)
SW[0]	Input	PIN_AB12	3A	B3A_N0	PIN_AB12	2.5 V		12mA (default)
ADC_CS_N	Unknown	PIN_AJ4	3B	B3B_N0		3.3-V LVTTL		16mA (default)

Figure 21b. Pin planner table.

Code

Shift Register

```
// synopsys translate_off
`timescale 1 ps / 1 ps
// synopsys translate_on
module shiftregister (
	clock,
	shiftin,
	shiftout,
	taps);

	input	  clock;
	input	[15:0]  shiftin;
	output	[15:0]  shiftout;
	output	[511:0]  taps;

	wire [15:0] sub_wire0;
	wire [511:0] sub_wire1;
	wire [15:0] shiftout = sub_wire0[15:0];
	wire [511:0] taps = sub_wire1[511:0];

	altshift_taps	ALTSHIFT_TAPS_component (
				.clock (clock),
				.shiftin (shiftin),
				.shiftout (sub_wire0),
				.taps (sub_wire1)
				// synopsys translate_off
				,
				.aclr (),
				.clken (),
				.sclr ()
				// synopsys translate_on
				);
	defparam
		ALTSHIFT_TAPS_component.intended_device_family = "Cyclone V",
		ALTSHIFT_TAPS_component.lpm_hint = "RAM_BLOCK_TYPE=MLAB",
		ALTSHIFT_TAPS_component.lpm_type = "altshift_taps",
		ALTSHIFT_TAPS_component.number_of_taps = 64,
		ALTSHIFT_TAPS_component.tap_distance = 30,
		ALTSHIFT_TAPS_component.width = 16;
endmodule
```

Echo Machine

```
module echo_machine (
	input  wire clk,
	input  wire reset,
	input  wire signed [15:0] x_in,
	output reg  signed [15:0] y_out
);

	wire signed [15:0] delayed_sample;

	shiftregister u_delay (
		.clock (clk),
		.shiftin (x_in),
		.shiftout(delayed_sample)
	);

	// divider
	wire signed [15:0] scaled_echo;
	assign scaled_echo = delayed_sample >>> 2;

	// adder
	wire signed [18:0] sum; //allows for overflow cases
	assign sum = x_in + scaled_echo;

	// output register
	always @(posedge clk or posedge reset) begin
		if (reset) begin
			y_out <= 16'sd0;
		end
		else begin
			y_out <= sum[15:0];  // truncated to 16-bits
		end
	end

endmodule // echo_machine
```

Multiplier

```
// synopsys translate_off
`timescale 1 ps / 1 ps
// synopsys translate_on
module multiplier (
	dataa,
	datab,
	result);

	input	[15:0]  dataa;
	input	[15:0]  datab;
	output	[31:0]  result;

	wire [31:0] sub_wire0;
	wire [31:0] result = sub_wire0[31:0];

	lpm_mult	lpm_mult_component (
				.dataa (dataa),
				.datab (datab),
				.result (sub_wire0),
				.aclr (1'b0),
				.clken (1'b1),
				.clock (1'b0),
				.sclr (1'b0),
				.sum (1'b0));
	defparam
		lpm_mult_component.lpm_hint = "MAXIMIZE_SPEED=5",
		lpm_mult_component.lpm_representation = "SIGNED",
		lpm_mult_component.lpm_type = "LPM_MULT",
		lpm_mult_component.lpm_widtha = 16,
		lpm_mult_component.lpm_widthb = 16,
		lpm_mult_component.lpm_widthp = 32;


endmodule
```

FIR Filter

```
module fir_filter(
	input  wire clk,	// sample clock (8 kHz)
	input  wire reset,	// active-high reset
	input  wire signed [15:0] x_in,	// input sample
	output reg  signed [15:0] y_out	// output sample
);
	parameter TAPS = 65;  // default value 65

	// shift register for storing delays shifting to the right
	reg signed [15:0] x_delay [0:TAPS-1];

	// store products from multiplier
	wire signed [31:0] prod [0:TAPS-1];

	integer i;
	always @(posedge clk or posedge reset) begin
		if (reset) begin // active-high reset
			for (i=0 ; i<TAPS ; i=i+1) begin
				x_delay[i] <= 16'sd0;
			end
		end
		else begin
			x_delay[0] <= x_in; // save
			for (i=1 ; i<TAPS ; i=i+1) begin
				x_delay[i] <= x_delay[i-1]; //shift down
			end
		end
	end

	// assign coefficients for MATLAB integers after x 32768

	reg signed [15:0] coeff [0:TAPS-1];

	integer k;
	always @(*) begin
		// default all zeros to avoid latches / unknowns
		coeff[0]=	-6;
		coeff[1]=	-0;
		coeff[2]=	 18;
		coeff[3]=	  0;
		coeff[4]=	-43;
		coeff[5]=	 -0;
		coeff[6]=	 87;
```

```
coeff[7]=      0;
coeff[8]=   -158;
coeff[9]=     -0;
coeff[10]=    261;
coeff[11]=      0;
coeff[12]=   -403;
coeff[13]=     -0;
coeff[14]=    585;
coeff[15]=      0;
coeff[16]=   -807;
coeff[17]=     -0;
coeff[18]=   1061;
coeff[19]=      0;
coeff[20]=  -1338;
coeff[21]=     -0;
coeff[22]=   1620;
coeff[23]=      0;
coeff[24]=  -1890;
coeff[25]=     -0;
coeff[26]=   2127;
coeff[27]=      0;
coeff[28]=  -2313;
coeff[29]=     -0;
coeff[30]=   2431;
coeff[31]=      0;
coeff[32]=  30296;
coeff[33]=      0;
coeff[34]=   2431;
coeff[35]=     -0;
coeff[36]=  -2313;
coeff[37]=      0;
coeff[38]=   2127;
coeff[39]=     -0;
coeff[40]=  -1890;
coeff[41]=      0;
coeff[42]=   1620;
coeff[43]=     -0;
coeff[44]=  -1338;
coeff[45]=      0;
coeff[46]=   1061;
coeff[47]=     -0;
coeff[48]=   -807;
coeff[49]=      0;
coeff[50]=    585;
```

```
        coeff[51]=       -0;
        coeff[52]=     -403;
        coeff[53]=        0;
        coeff[54]=      261;
        coeff[55]=       -0;
        coeff[56]=     -158;
        coeff[57]=        0;
        coeff[58]=       87;
        coeff[59]=       -0;
        coeff[60]=      -43;
        coeff[61]=        0;
        coeff[62]=       18;
        coeff[63]=       -0;
        coeff[64]=       -6;
end

// 16x16 signed to 32-bit signed product of coefficients multiplied  by 32768
genvar t;
generate
        for (t = 0; t < TAPS; t = t + 1) begin : GEN_MUL
                multiplier u_mult (
                        .dataa(x_delay[t]),  // 16-bit signed
                        .datab(coeff[t]),        // 16-bit signed
                        .result(prod[t])      // 16-bit signed
                );
        end
endgenerate

// summing all true coefficients together(adder built in)
reg signed [31:0] sum;
integer s;
always @(*) begin
        sum = 32'sd0;
        for (s=0 ; s<TAPS ; s=s+1)
                // summing together all products
                sum = sum + prod[s];
end

// divide by 2^15 (32768)
wire signed [31:0] sum_scaled;
assign sum_scaled = sum >>> 15;

// output register per clock cycle
always @(posedge clk or posedge reset) begin
```

```
                if (reset) begin
                        y_out <= 16'sd0;
                end
                else begin
                        y_out <= sum_scaled[15:0]; // summed scaled sum truncated to 16 bits
                end
        end

endmodule // fir_filter
```

Multiplexer

```
module mux3_1(input signed [15:0] og_signal, fir, echo,
                                        input [1:0] sel,
                                        output reg signed [15:0] out);

        always @(*) begin
                case (sel)
                        2'b00: out = og_signal;
                        2'b01: out = fir;
                        2'b10: out = echo;
                        default: out = og_signal;
                endcase
        end
endmodule // mux3_1
```

DSP Subsystem

```
module dsp_subsystem (
	input wire sample_clock,
	input wire reset,
	input wire [1:0] selector,
	input wire signed [15:0] input_sample,
	output signed [15:0] output_sample);

	wire signed [15:0] y_fir;
	wire signed [15:0] y_echo;
	wire signed [15:0] mux_out;

	// FIR filter
	fir_filter u_fir (
			.clk (sample_clock),
			.reset (reset),
			.x_in (input_sample),
			.y_out(y_fir)
	);
	defparam u_fir.TAPS = 65;

	// echo machine
	echo_machine u_echo (
			.clk (sample_clock),
			.reset (reset),
			.x_in (input_sample),
			.y_out (y_echo)
	);

	// 3-to-1 multiplexer
	mux3_1 mux (
			.og_signal(input_sample),
			.fir(y_fir),
			.echo(y_echo),
			.sel(selector),
			.out(mux_out)
	);

	assign output_sample = mux_out;
endmodule // dsp_subsystem
```